

XGBoost Classification model

MAKINI CALEB

2026-10-01

SETTING A WORKING DIRECTORY

```
setwd("C:/Users/ADMIN/OneDrive/Desktop/caleb makini")
```

loading libraries

```
library(xgboost)
library(rio)
library(caret)
library(ggplot2)
library(dplyr)
library(rio)
library(pROC)
```

LOAD DATA SET

```
health_data <- read.csv("diabetes.csv")
head(health_data)
```

	age	hypertension	heart_disease	smoking_history	BMI	HbA1c_level
1	34	0	0	never	24.5	5.2
2	58	1	0	former	28.1	6.1
3	42	0	0	current	31.4	7.8
4	65	1	1	former	26.7	5.9
5	29	0	0	never	22.1	4.9
6	71	1	0	never	29.3	6.8

	blood_glucose_level	diabetes
1	89	0
2	112	0
3	145	1
4	98	0
5	82	0
6	132	1

check for missing values

```
health_data <- na.omit(health_data)
colSums((is.na(health_data)))
```

	age	hypertension	heart_disease	smokin
g_history	0	0	0	
0				
	BMI	HbA1c_level	blood_glucose_level	
diabetes	0	0	0	
0				

confirm for missing values

```
colSums((is.na(health_data)))
```

	age	hypertension	heart_disease	smokin
g_history	0	0	0	
0				
	BMI	HbA1c_level	blood_glucose_level	
diabetes	0	0	0	
0				

structure

```
str(health_data)
```

```
'data.frame': 1099 obs. of 8 variables:
 $ age          : int  34 58 42 65 29 71 38 55 44 62 ...
 $ hypertension : int  0 1 0 1 0 1 0 1 0 1 ...
 $ heart_disease : int  0 0 0 1 0 0 0 1 0 0 ...
 $ smoking_history : chr  "never" "former" "current" "former" ...
 $ BMI          : num  24.5 28.1 31.4 26.7 22.1 29.3 33.2 27.9 2
5.6 30.1 ...
 $ HbA1c_level   : num  5.2 6.1 7.8 5.9 4.9 6.8 8.5 6.3 5.4 7.1
...
 $ blood_glucose_level: int  89 112 145 98 82 132 162 118 91 138 ...
 $ diabetes       : int  0 0 1 0 0 1 1 0 0 1 ...
- attr(*, "na.action")= 'omit' Named int 1100
... attr(*, "names")= chr "1100"
```

```
table(health_data$diabetes)
```

```
  0   1  
612 487
```

convert categorical variables into factor

```
health_data$diabetes <- as.factor(health_data$diabetes)
```

split the data set

```
set.seed(123)  
trainIndexed <- createDataPartition(health_data$diabetes, p = 0.8, list = FALSE)  
train <- health_data[trainIndexed, ]  
test <- health_data[-trainIndexed, ]
```

one hot encoding variables

```
dummy <- dummyVars(diabetes ~ ., data = train)  
train_x <- predict(dummy, train)  
test_x <- predict(dummy, test)  
train_y <- as.numeric(as.character(train$diabetes))  
test_y <- as.numeric(as.character(test$diabetes))
```

create dmatrix

```
dtrain <- xgb.DMatrix(data = train_x, label = train_y)  
dtest <- xgb.DMatrix(data = test_x, label = test_y)
```

train xgboost model

```
params <- list(objective = "binary:logistic", eval_metric = "logloss",  
max_depth = 4,  
eta = 0.1, subsample = 0.8, colsample_bytree = 0.8)  
xgb_model <- xgb.train(params = params, data = dtrain, nrounds = 100,  
watchlist = list(train = dtrain),  
verbose = 1)
```

```
[1] train-logloss:0.596029
[2] train-logloss:0.531379
[3] train-logloss:0.468267
[4] train-logloss:0.417471
[5] train-logloss:0.371432
[6] train-logloss:0.332137
[7] train-logloss:0.303272
[8] train-logloss:0.272472
[9] train-logloss:0.246919
[10]   train-logloss:0.224276
[11]   train-logloss:0.202818
[12]   train-logloss:0.183925
[13]   train-logloss:0.168487
[14]   train-logloss:0.153170
[15]   train-logloss:0.139485
[16]   train-logloss:0.129947
[17]   train-logloss:0.118257
[18]   train-logloss:0.107960
[19]   train-logloss:0.099280
[20]   train-logloss:0.091921
[21]   train-logloss:0.084452
[22]   train-logloss:0.078161
[23]   train-logloss:0.071850
[24]   train-logloss:0.066497
[25]   train-logloss:0.062365
[26]   train-logloss:0.057473
[27]   train-logloss:0.053182
[28]   train-logloss:0.049343
[29]   train-logloss:0.046186
[30]   train-logloss:0.042945
[31]   train-logloss:0.040063
[32]   train-logloss:0.037322
[33]   train-logloss:0.034977
[34]   train-logloss:0.033112
[35]   train-logloss:0.031187
[36]   train-logloss:0.029680
[37]   train-logloss:0.028231
[38]   train-logloss:0.026503
[39]   train-logloss:0.025221
[40]   train-logloss:0.024004
[41]   train-logloss:0.022807
[42]   train-logloss:0.021670
[43]   train-logloss:0.020716
[44]   train-logloss:0.019963
[45]   train-logloss:0.019103
[46]   train-logloss:0.018316
[47]   train-logloss:0.017696
[48]   train-logloss:0.017038
[49]   train-logloss:0.016529
```

```
[50]    train-logloss:0.015975
[51]    train-logloss:0.015456
[52]    train-logloss:0.015014
[53]    train-logloss:0.014587
[54]    train-logloss:0.014181
[55]    train-logloss:0.013851
[56]    train-logloss:0.013426
[57]    train-logloss:0.013031
[58]    train-logloss:0.012714
[59]    train-logloss:0.012347
[60]    train-logloss:0.012070
[61]    train-logloss:0.011703
[62]    train-logloss:0.011449
[63]    train-logloss:0.011264
[64]    train-logloss:0.011010
[65]    train-logloss:0.010767
[66]    train-logloss:0.010555
[67]    train-logloss:0.010376
[68]    train-logloss:0.010233
[69]    train-logloss:0.010051
[70]    train-logloss:0.009923
[71]    train-logloss:0.009700
[72]    train-logloss:0.009553
[73]    train-logloss:0.009447
[74]    train-logloss:0.009307
[75]    train-logloss:0.009116
[76]    train-logloss:0.008994
[77]    train-logloss:0.008870
[78]    train-logloss:0.008738
[79]    train-logloss:0.008646
[80]    train-logloss:0.008524
[81]    train-logloss:0.008494
[82]    train-logloss:0.008398
[83]    train-logloss:0.008282
[84]    train-logloss:0.008255
[85]    train-logloss:0.008191
[86]    train-logloss:0.008077
[87]    train-logloss:0.007997
[88]    train-logloss:0.007930
[89]    train-logloss:0.007835
[90]    train-logloss:0.007786
[91]    train-logloss:0.007701
[92]    train-logloss:0.007621
[93]    train-logloss:0.007544
[94]    train-logloss:0.007442
[95]    train-logloss:0.007343
[96]    train-logloss:0.007281
[97]    train-logloss:0.007216
[98]    train-logloss:0.007148
```

```
[99]    train-logloss:0.007084
[100]   train-logloss:0.007018
```

make predictions

```
make_predictions <- predict(xgb_model, dtest)
predictions <- ifelse(make_predictions > 0.5, 1, 0)
```

confusion matrix

```
confusionMatrix(factor(predictions), factor(test_y))
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	122	2
1	0	95

Accuracy : 0.9909

95% CI : (0.9674, 0.9989)

No Information Rate : 0.5571

P-Value [Acc > NIR] : <2e-16

Kappa : 0.9815

Mcnemar's Test P-Value : 0.4795

Sensitivity : 1.0000

Specificity : 0.9794

Pos Pred Value : 0.9839

Neg Pred Value : 1.0000

Prevalence : 0.5571

Detection Rate : 0.5571

Detection Prevalence : 0.5662

Balanced Accuracy : 0.9897

'Positive' Class : 0

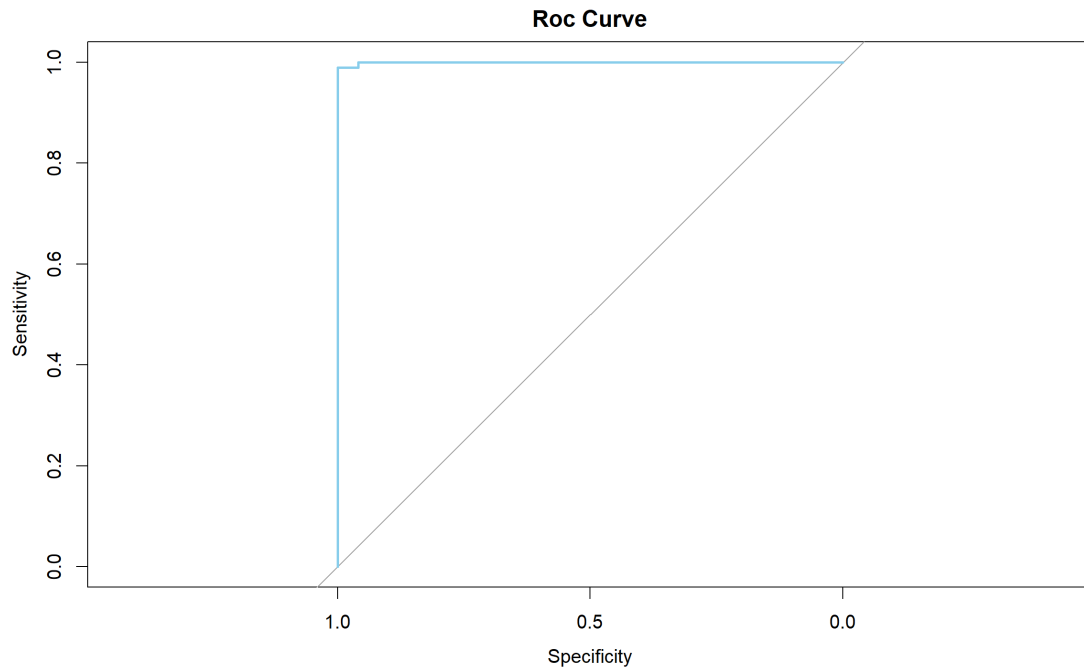
- The Confusion Matrix (The Raw Numbers)A confusion matrix compares what the model predicted against the actual reference truth. The dataset has 219 total samples, split by the model like this: True Positives (122): The actual class was 0, and the model correctly predicted 0. True Negatives (95): The actual class was 1, and the model correctly predicted 1.False Positives (2): The actual class was 1, but the model

incorrectly guessed 0. False Negatives (0): The actual class was 0, and the model never missed it.

- Overall Model Performance Metrics** These metrics evaluate how well the model works as a whole across all data points:
 - Accuracy (99.09%)**: The overall success rate. The model guessed correctly 217 out of 219 times.
 - 95% Confidence Interval (96.74%, 99.89%)**: A statistical range. If you ran this model on new, identical distributions of data, you can be 95% confident the accuracy would fall between these two percentages.
 - No Information Rate / NIR (55.71%)**: The accuracy you would get by blindly guessing the majority class (Class 0) every single time.
 - P-Value [Acc > NIR] (< 2e-16)**: A measure of statistical significance. Because it is essentially zero, it proves your model's high accuracy is due to genuine predictive power, not random luck.
 - Kappa (0.9815)**: Measures agreement between predictions and reality, adjusted for chance. A score above 0.80 is considered excellent; 0.98 indicates near-perfect reliability.
 - McNemar's Test P-Value (0.4795)**: Checks if the model's errors are heavily biased toward one type of mistake over another. A high p-value (greater than 0.05) means your small number of misclassifications (just 2 errors) are statistically balanced.
- Class-Specific Metrics (Class 0 as 'Positive')** Because Class 0 is designated as the target "Positive" class, the performance breaks down into specific dimensions of accuracy:
 - Sensitivity / Recall (100.00%)**: Out of all actual Class 0 cases in the data, the model found every single one. It missed zero positive cases.
 - Specificity (97.94%)**: Out of all actual Class 1 cases, the model correctly identified 97.94% of them. It only misclassified two.
 - Balanced Accuracy (98.97%)**: The average of Sensitivity and Specificity. It gives an honest view of performance when classes are unevenly split.
 - Positive Predictive Value / Precision (98.39%)**: When the model claims a sample is Class 0, it is right 98.39% of the time.
 - Negative Predictive Value (100.00%)**: When the model claims a sample is Class 1, it is right every single time.
 - Data Split & Prevalence**
 - Prevalence (55.71%)**: The actual proportion of Class 0 in your entire dataset ($\frac{122}{219}$).
 - Detection Rate (55.71%)**: The rate at which the model successfully found true positives relative to the total dataset size.
 - Detection Prevalence (56.62%)**: The total proportion of the dataset that the model claimed belonged to Class 0, regardless of whether it was right or wrong ($\frac{124}{219}$).
- Formatting Styles Explained** When putting this data into a final report, the writing changes drastically based on who is reading it:
 - Academic / APA Style**: Written for researchers, peer reviews, or journals. It relies heavily on parenthetical statistics, technical terminology, and exact numerical representations (e.g., "The classifier demonstrated a significant improvement over the baseline No Information Rate, $\chi^2 \dots p < .001 \dots$ ").
 - Business / Executive Style**: Written for project stakeholders, managers, or clients. It strips away technical jargon (like Kappa or McNemar's) and frames the numbers around risk and operational reliability (e.g., "Our system operates at a 99.09% success rate. Crucially, it has a 0% fail-miss rate for Class 0, meaning no critical targets are bypassed...").

ROC_CURVE

```
roc_curve <- roc(test_y, make_predictions)
plot(roc_curve, col = "skyblue", main = "Roc Curve")
```



visualize diabetes

```
ggplot(health_data, aes(x = factor(diabetes))) + geom_bar(fill = "yellowgreen") +
  labs(title = "Distribution of diabetes", x = "Diabetes", y = "Frequency")
```

Diustribution of diabetes

